

EDA ANALYSIS

1. Basic Statistical Summary

In [2]: `import pandas as pd`

```
# Load dataset
df = pd.read_csv(r"C:\Users\Vinay Kumar\Downloads\cleaned_data.csv")

# Summary statistics
df.describe()
```

Out[2]:

	Order_ID	Customer_ID	Product_ID	Original_Price	Discount_%	Qual
count	22000.000000	22000.000000	22000.000000	22000.000000	22000.000000	22000.00
mean	11000.500000	1048.991227	230.084682	1049.581127	12.922273	2.54
std	6350.997297	29.685395	17.202507	556.978784	9.896912	1.11
min	1.000000	1000.000000	200.000000	95.930000	0.000000	1.00
25%	5500.750000	1023.000000	216.000000	548.922500	5.000000	2.00
50%	11000.500000	1050.000000	231.000000	1061.435000	10.000000	3.00
75%	16500.250000	1074.000000	245.000000	1536.567500	20.000000	4.00
max	22000.000000	1099.000000	259.000000	2096.170000	30.000000	4.00



2. Central Tendency Measures

In [3]:

```
# Mean
print("Mean Revenue:", df['Revenue_Check'].mean())

# Median
print("Median Revenue:", df['Revenue_Check'].median())

# Mode
print("Mode Revenue:", df['Revenue_Check'].mode()[0])
```

Mean Revenue: 2324.150255

Median Revenue: 1876.7350000000001

Mode Revenue: 2202.84

The average (mean) revenue of ₹2324 indicates that, on average, each transaction generates around this amount. However, the median revenue is lower at ₹1876, which means that half of the transactions are below ₹1876 and half are above it. This difference between the mean and median suggests that the data is slightly positively skewed, meaning there are some high-value transactions pulling the average upward. The mode revenue, ₹2202.84, represents the most frequently occurring transaction value, showing a

common purchase level among customers. Overall, this pattern indicates that while most customers make moderate purchases, a smaller group of customers contributes higher revenue, leading to variation in spending behavior. This highlights the presence of different customer segments, such as regular buyers and high-value customers, which is important for targeted marketing and business strategy.

3. Variability / Spread

```
In [6]: # Variance
print("Variance:", df['Revenue_Check'].var())

# Standard Deviation
print("Std Dev:", df['Revenue_Check'].std())

# Range
print("Range:", df['Revenue_Check'].max() - df['Revenue_Check'].min())
```

Variance: 2907592.2403120827

Std Dev: 1705.166338018694

Range: 8247.98

The variance of 2907592 and standard deviation of ₹1705 indicate that there is a high level of variability in the revenue data. In simple terms, customer spending is not consistent—some customers spend much less, while others spend significantly more. The standard deviation shows that, on average, individual transaction values differ from the mean (₹2324) by about ₹1705, which is quite large. Additionally, the range of ₹8247.98 (difference between maximum and minimum revenue) further confirms that there is a wide gap between the lowest and highest transactions. Overall, these measures highlight that the business is dealing with diverse customer segments, including low spenders and high-value customers, which creates fluctuations in revenue patterns.

4. Skewness & Kurtosis

```
In [7]: # Skewness
print("Skewness:", df['Revenue_Check'].skew())

# Kurtosis
print("Kurtosis:", df['Revenue_Check'].kurt())
```

Skewness: 0.8427610257467393

Kurtosis: -0.06353616597692024

The skewness value of 0.84 indicates that the data is positively skewed (right-skewed). This means most of the revenue values are concentrated on the lower side, while a few high-value transactions pull the distribution toward the right. In practical terms, most customers are spending moderate or low amounts, but a small number of customers are making significantly higher purchases.

The kurtosis value of -0.06 is close to zero, which means the distribution is approximately normal (mesokurtic). This suggests that the data does not have extreme outliers or

unusually sharp peaks. The spread of data is fairly balanced, without too many extreme values.

5. Correlation Analysis

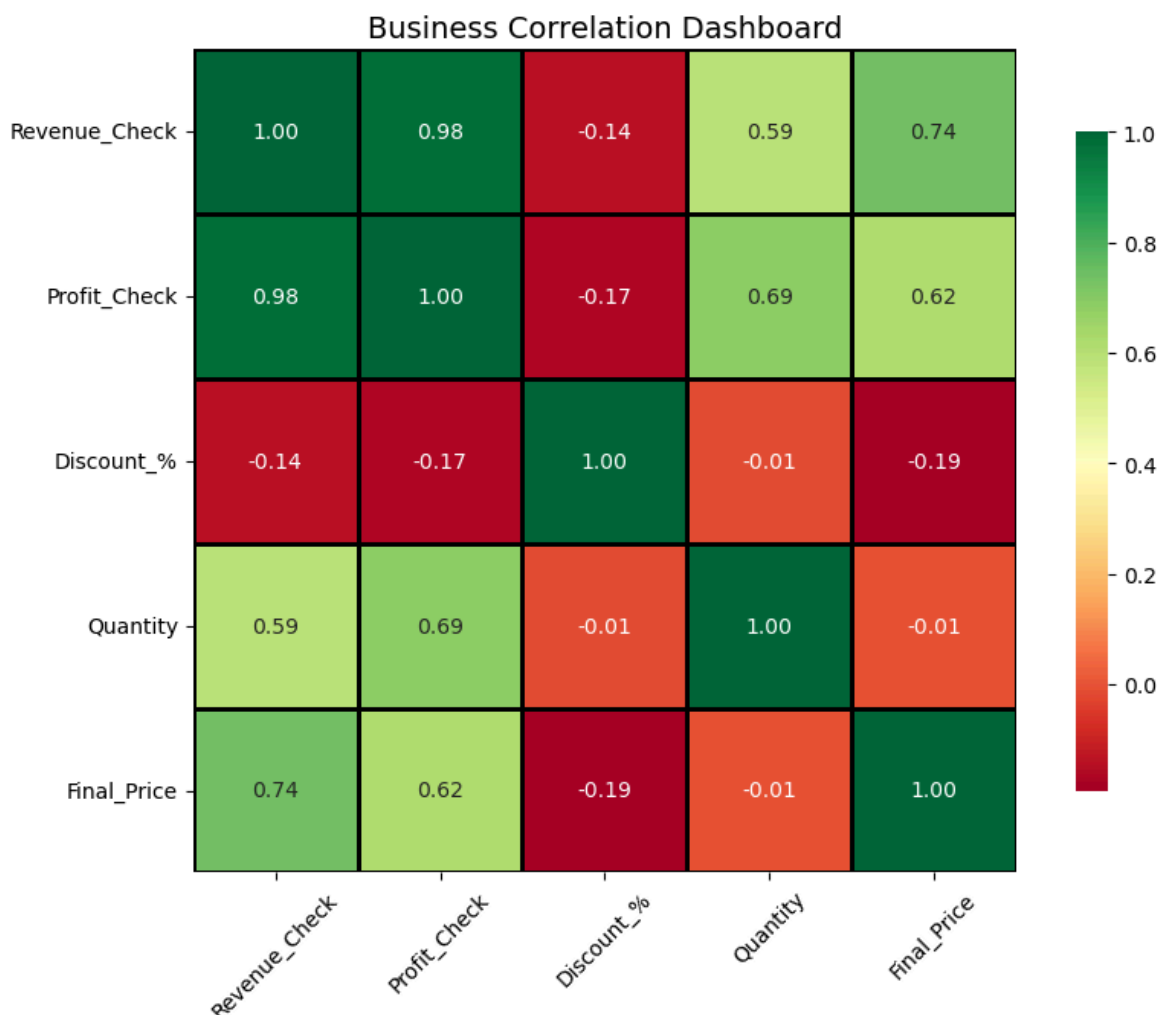
```
In [8]: # Correlation matrix
corr = df[['Revenue_Check', 'Profit_Check', 'Discount_%', 'Quantity', 'Final_Price']]
print(corr)
```

	Revenue_Check	Profit_Check	Discount_%	Quantity	Final_Price
Revenue_Check	1.000000	0.982273	-0.141325	0.589186	0.738612
Profit_Check	0.982273	1.000000	-0.165273	0.687140	0.616304
Discount_%	-0.141325	-0.165273	1.000000	-0.013311	-0.192059
Quantity	0.589186	0.687140	-0.013311	1.000000	-0.007516
Final_Price	0.738612	0.616304	-0.192059	-0.007516	1.000000

```
In [13]: import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [14]: plt.figure(figsize=(10,7))
sns.heatmap(
    corr,
    annot=True,
    cmap='RdYlGn',
    fmt='.2f',
    linewidths=1,
    linecolor='black',
    square=True,
    cbar_kws={"shrink": .8}
)

plt.title('Business Correlation Dashboard', fontsize=14)
plt.xticks(rotation=45)
plt.yticks(rotation=0)
plt.show()
```



6. Covariance (Relationship Strength)

```
In [15]: df[['Revenue_Check', 'Profit_Check']].cov()
```

```
Out[15]:
```

	Revenue_Check	Profit_Check
Revenue_Check	2.907592e+06	2.452355e+06
Profit_Check	2.452355e+06	2.143723e+06

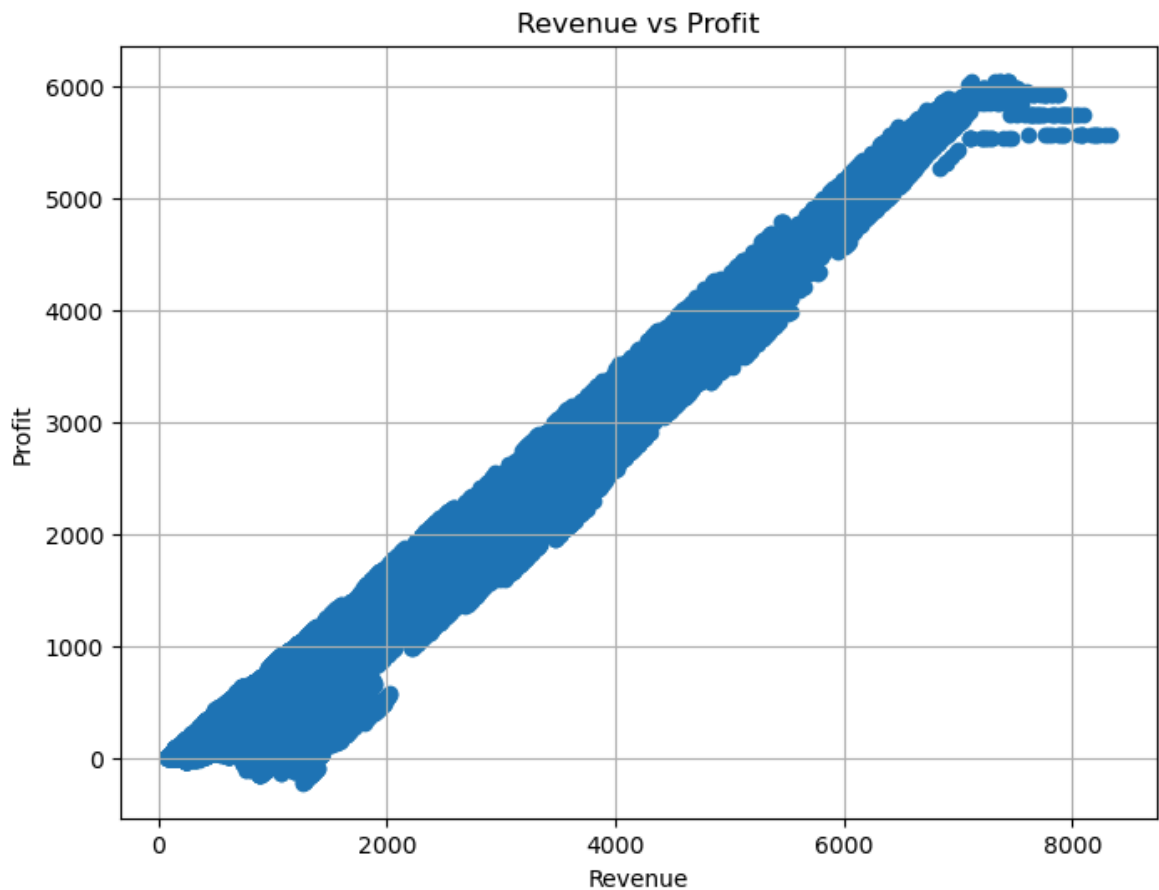
The covariance between revenue and profit is positive and high, indicating that both variables increase together. However, due to its scale dependency, covariance alone is difficult to interpret. Therefore, correlation analysis is more reliable, which confirms a strong positive relationship (0.98) between revenue and profit.

```
In [16]: import matplotlib.pyplot as plt

plt.figure(figsize=(8,6))
plt.scatter(df['Revenue_Check'], df['Profit_Check'])

plt.title("Revenue vs Profit")
plt.xlabel("Revenue")
plt.ylabel("Profit")
```

```
plt.grid()
plt.show()
```



7. Group-Based Statistics

```
In [18]: df.groupby('Category')['Revenue_Check'].mean()
```

```
Out[18]: Category
Beauty      2327.356740
Clothing    2416.141817
Electronics 2263.138904
Home        2279.568605
Name: Revenue_Check, dtype: float64
```

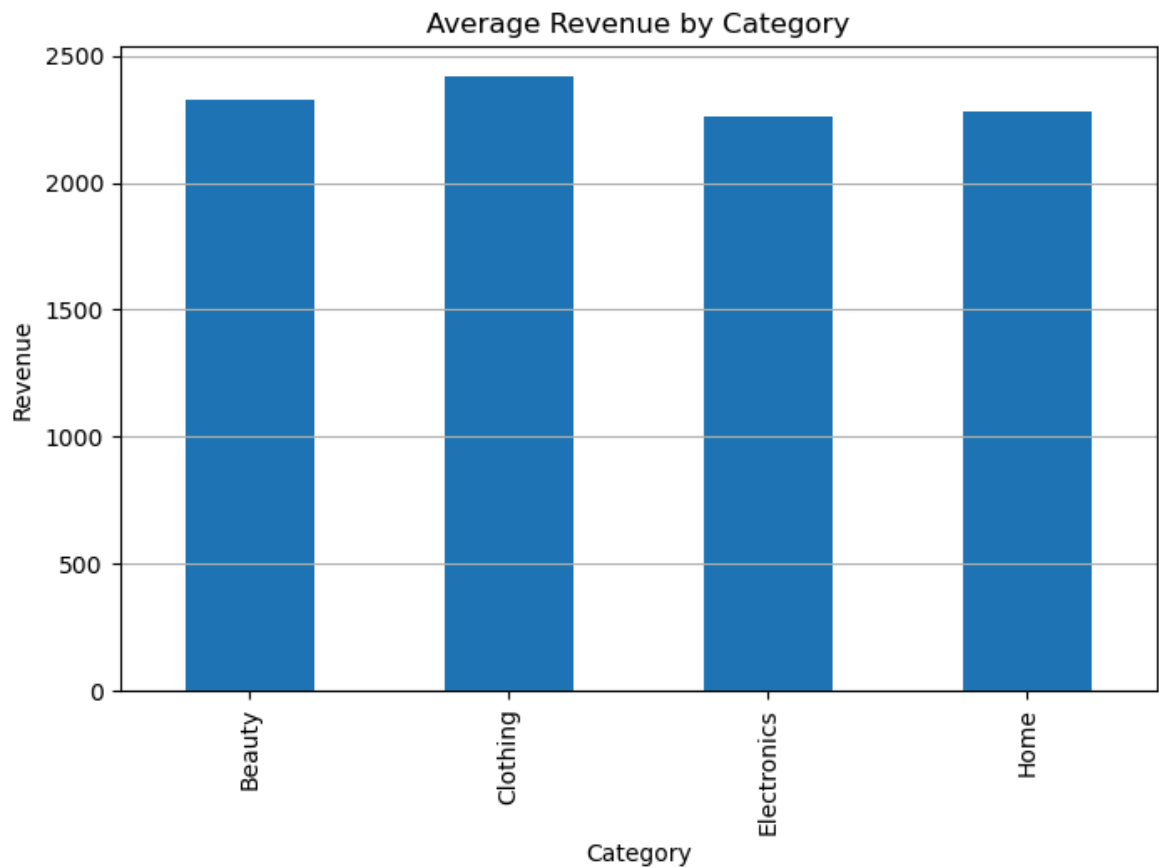
```
In [21]: import matplotlib.pyplot as plt

cat_rev = df.groupby('Category')['Revenue_Check'].mean()

plt.figure(figsize=(8,5))
cat_rev.plot(kind='bar')

plt.title("Average Revenue by Category")
plt.xlabel("Category")
plt.ylabel("Revenue")

plt.grid(axis='y')
plt.show()
```



```
In [19]: df.groupby('Region')['Profit_Check'].mean()
```

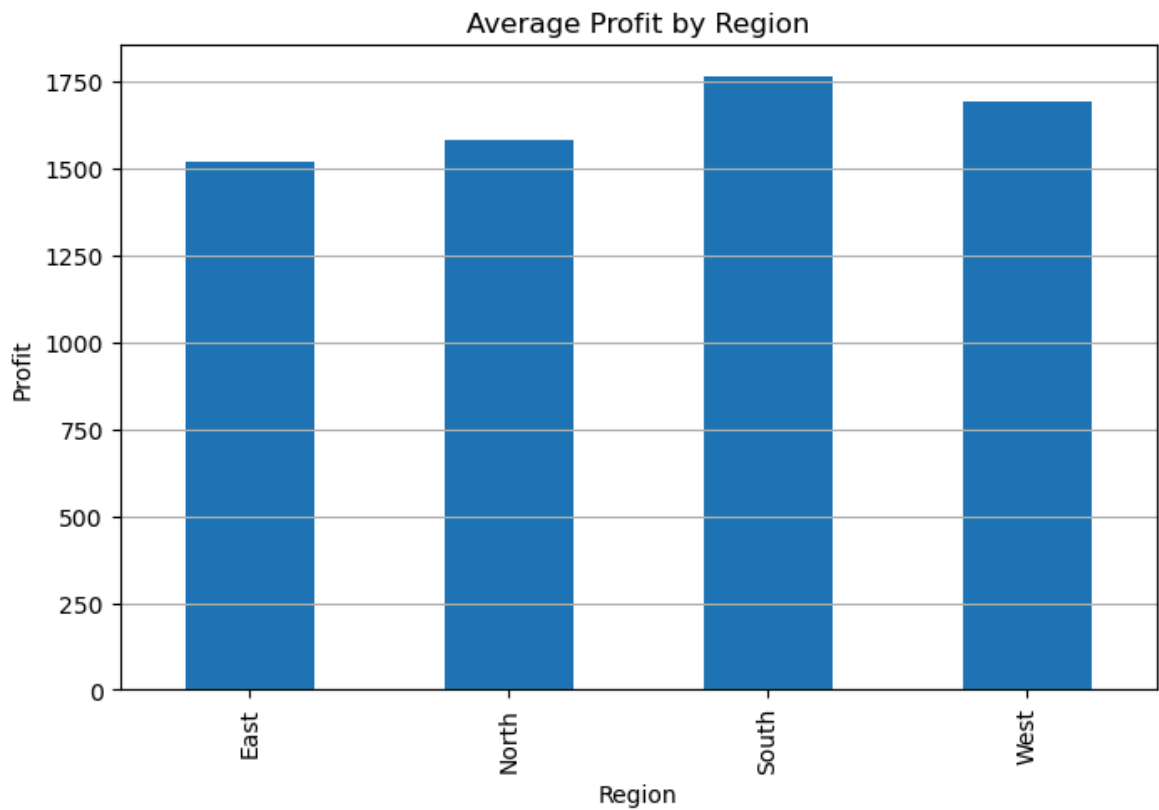
```
Out[19]: Region
East      1520.394575
North     1579.818429
South     1764.517901
West      1689.664268
Name: Profit_Check, dtype: float64
```

```
In [22]: reg_profit = df.groupby('Region')['Profit_Check'].mean()

plt.figure(figsize=(8,5))
reg_profit.plot(kind='bar')

plt.title("Average Profit by Region")
plt.xlabel("Region")
plt.ylabel("Profit")

plt.grid(axis='y')
plt.show()
```



```
In [20]: df.groupby('Customer_Gender')['Revenue_Check'].mean()
```

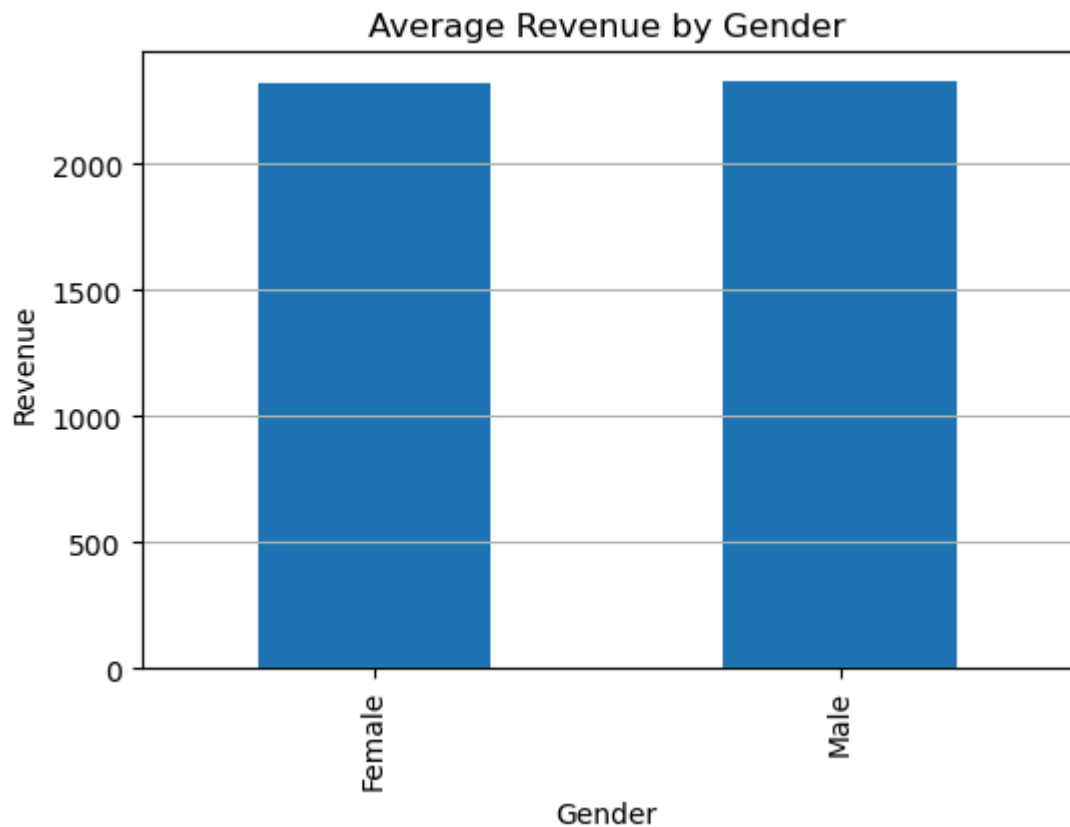
```
Out[20]: Customer_Gender
Female    2321.716256
Male      2326.505450
Name: Revenue_Check, dtype: float64
```

```
In [23]: gender_rev = df.groupby('Customer_Gender')['Revenue_Check'].mean()

plt.figure(figsize=(6,4))
gender_rev.plot(kind='bar')

plt.title("Average Revenue by Gender")
plt.xlabel("Gender")
plt.ylabel("Revenue")

plt.grid(axis='y')
plt.show()
```



8. Time-Based Statistics

```
In [24]: # Monthly revenue
df.groupby('Month')['Revenue_Check'].sum()

# Seasonal profit
df.groupby('Season')['Profit_Check'].mean()
```

```
Out[24]: Season
Festive    1735.461969
Regular    1619.652608
Name: Profit_Check, dtype: float64
```

```
In [25]: df.groupby('Month')['Revenue_Check'].sum()
```

```
Out[25]: Month
1      4893276.23
2      4432139.96
3      5109996.02
4      5066029.37
5      4093437.47
6      4488091.94
7      5198308.17
8      4311262.88
9      3880202.08
10     4002163.42
11     2975584.63
12     2680813.44
Name: Revenue_Check, dtype: float64
```

Monthly Revenue Trends Top Performing Months: July (7) → ₹51.98L March (3) → ₹51.09L April (4) → ₹50.66L Low Performing Months: December (12) → ₹26.80L November (11) → ₹29.75L Insight: Strong performance in mid-year (Mar–Jul) Sharp decline towards year-end

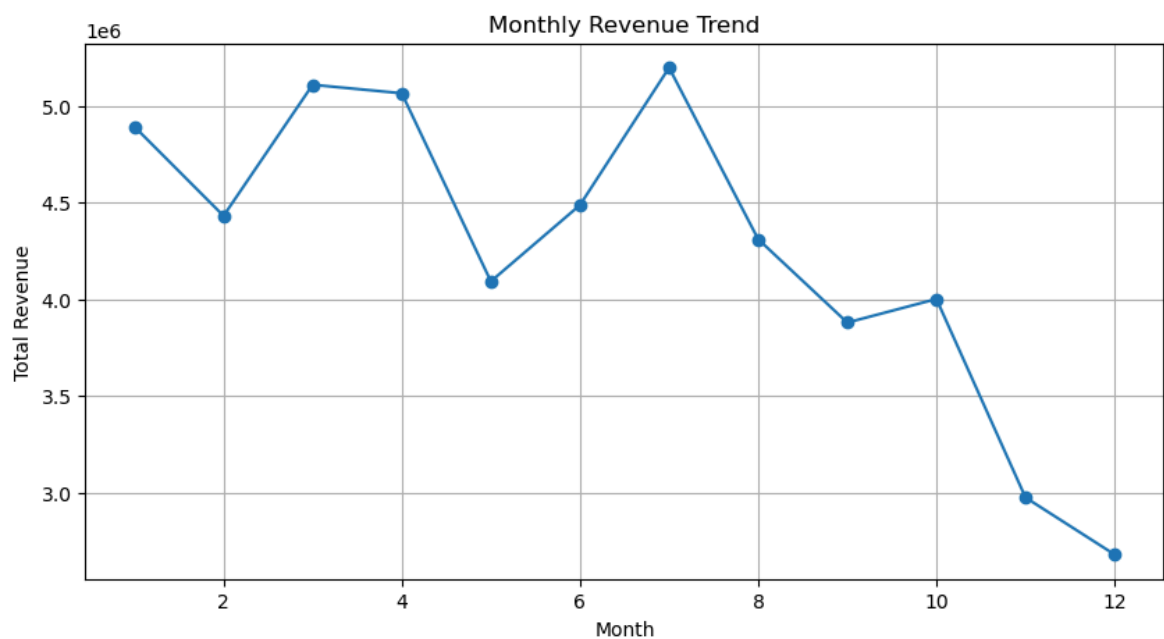
```
In [26]: import matplotlib.pyplot as plt

monthly_rev = df.groupby('Month')['Revenue_Check'].sum()

plt.figure(figsize=(10,5))
plt.plot(monthly_rev.index, monthly_rev.values, marker='o')

plt.title("Monthly Revenue Trend")
plt.xlabel("Month")
plt.ylabel("Total Revenue")

plt.grid()
plt.show()
```

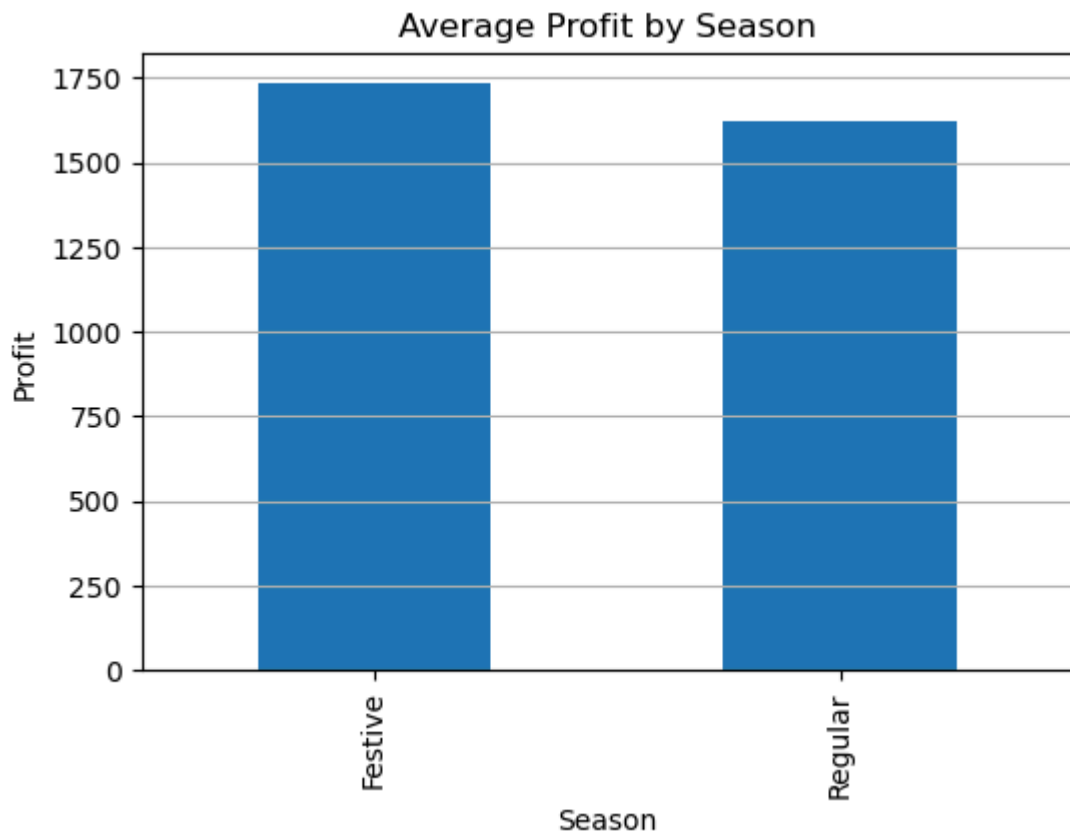


```
In [27]: season_profit = df.groupby('Season')['Profit_Check'].mean()

plt.figure(figsize=(6,4))
season_profit.plot(kind='bar')

plt.title("Average Profit by Season")
plt.xlabel("Season")
plt.ylabel("Profit")

plt.grid(axis='y')
plt.show()
```



9. Outlier Detection (Statistical)

```
In [28]: Q1 = df['Revenue_Check'].quantile(0.25)
Q3 = df['Revenue_Check'].quantile(0.75)
IQR = Q3 - Q1

outliers = df[(df['Revenue_Check'] < Q1 - 1.5*IQR) |
               (df['Revenue_Check'] > Q3 + 1.5*IQR)]

print("Outliers count:", len(outliers))
```

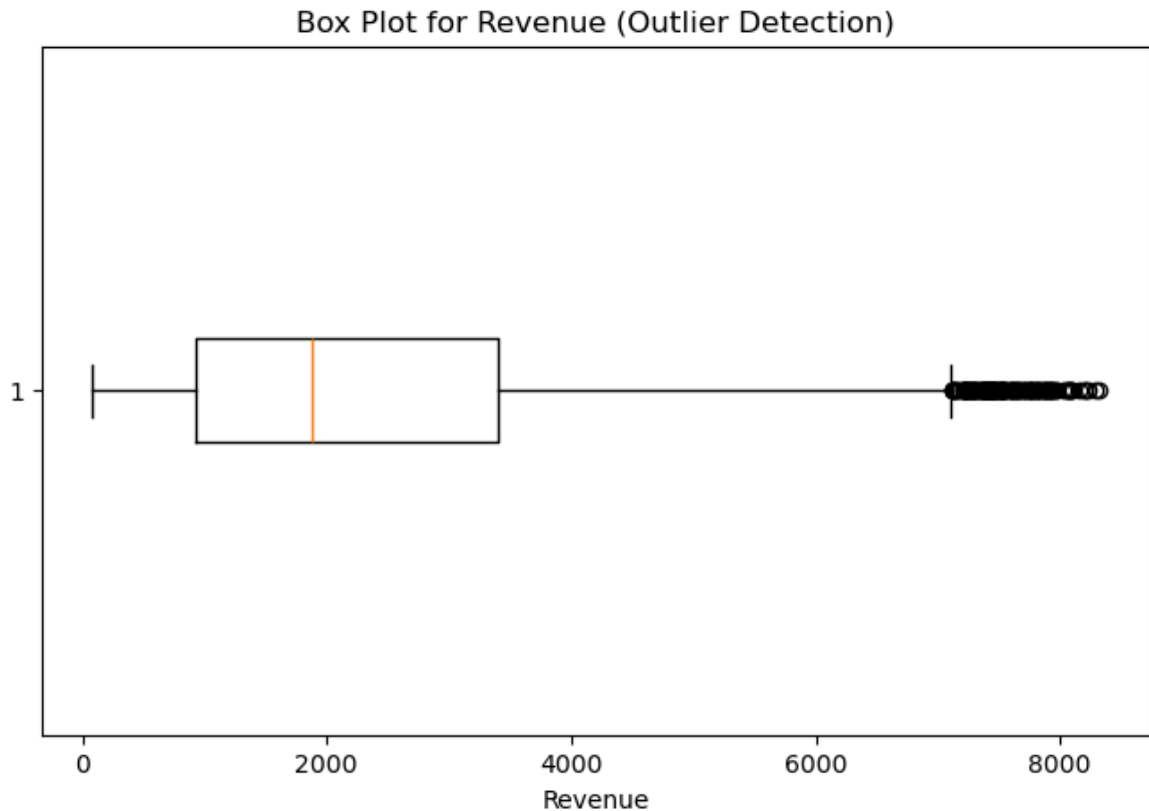
Outliers count: 136

```
In [31]: import matplotlib.pyplot as plt

plt.figure(figsize=(8,5))
plt.boxplot(df['Revenue_Check'], vert=False)

plt.title("Box Plot for Revenue (Outlier Detection)")
plt.xlabel("Revenue")

plt.show()
```



10. Loss-Making Transactions

```
In [29]: loss = df[df['Profit_Check'] < 0]

print("Loss transactions:", len(loss))
print(loss[['Category', 'Discount_%', 'Profit_Check']].head())
```

```
Loss transactions: 298
   Category  Discount_%  Profit_Check
74  Electronics      30.0      -63.69
132    Home          30.0      -30.17
145    Home          30.0     -124.83
227  Electronics      30.0      -50.61
398    Home          30.0     -53.90
```

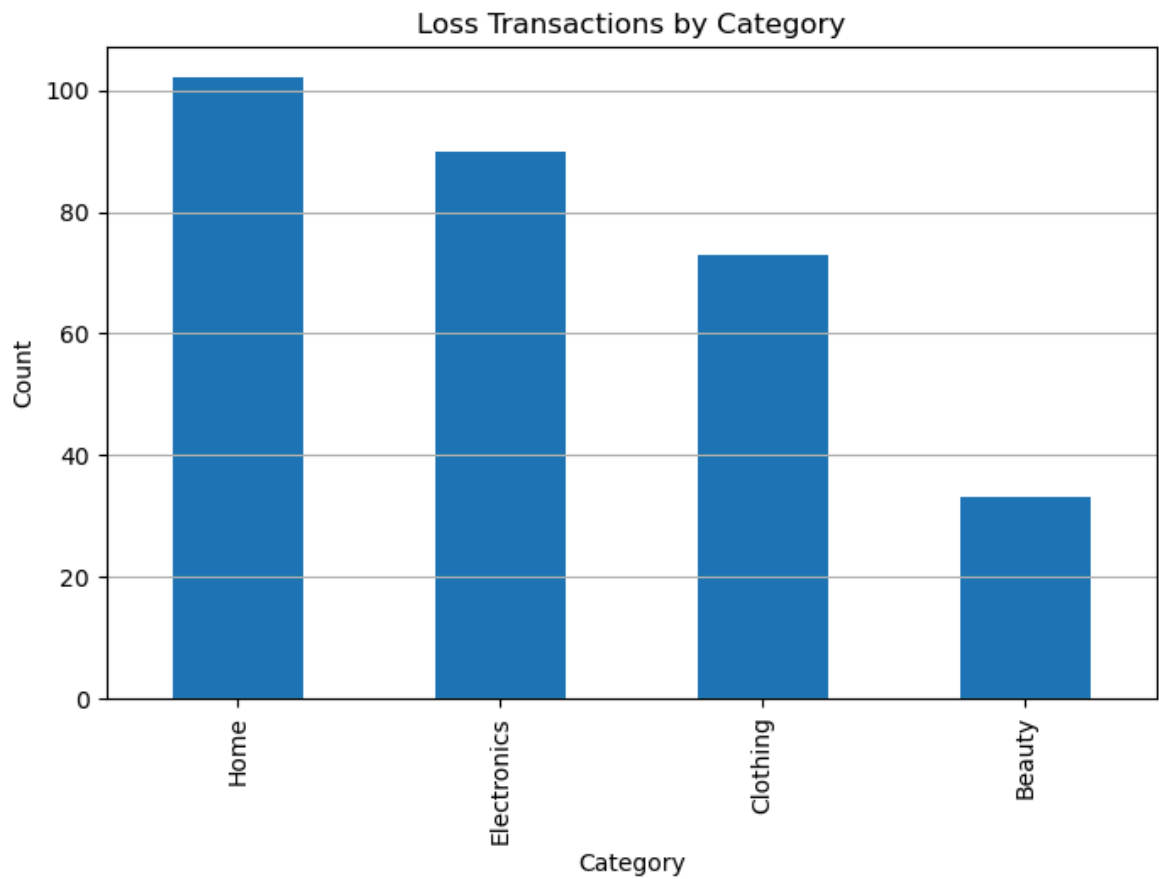
```
In [33]: import matplotlib.pyplot as plt

loss_cat = loss['Category'].value_counts()

plt.figure(figsize=(8,5))
loss_cat.plot(kind='bar')

plt.title("Loss Transactions by Category")
plt.xlabel("Category")
plt.ylabel("Count")

plt.grid(axis='y')
plt.show()
```

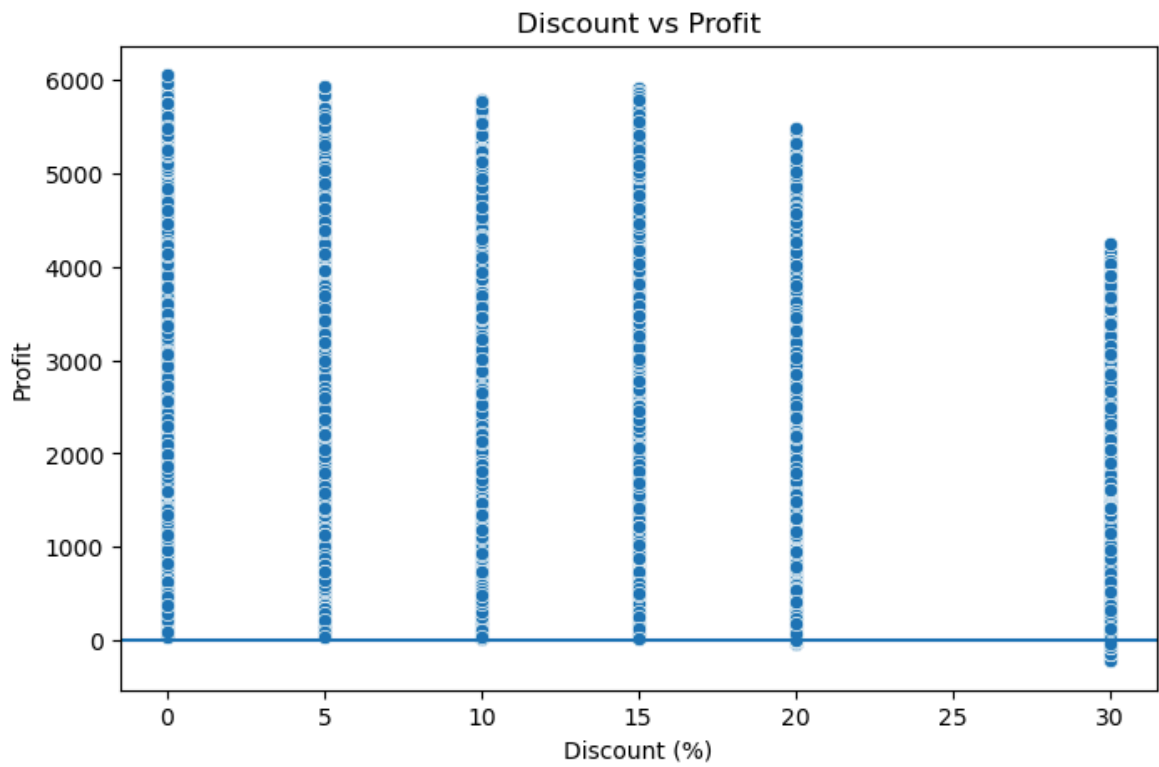


```
In [34]: import seaborn as sns

plt.figure(figsize=(8,5))
sns.scatterplot(x=df['Discount_%'], y=df['Profit_Check'])

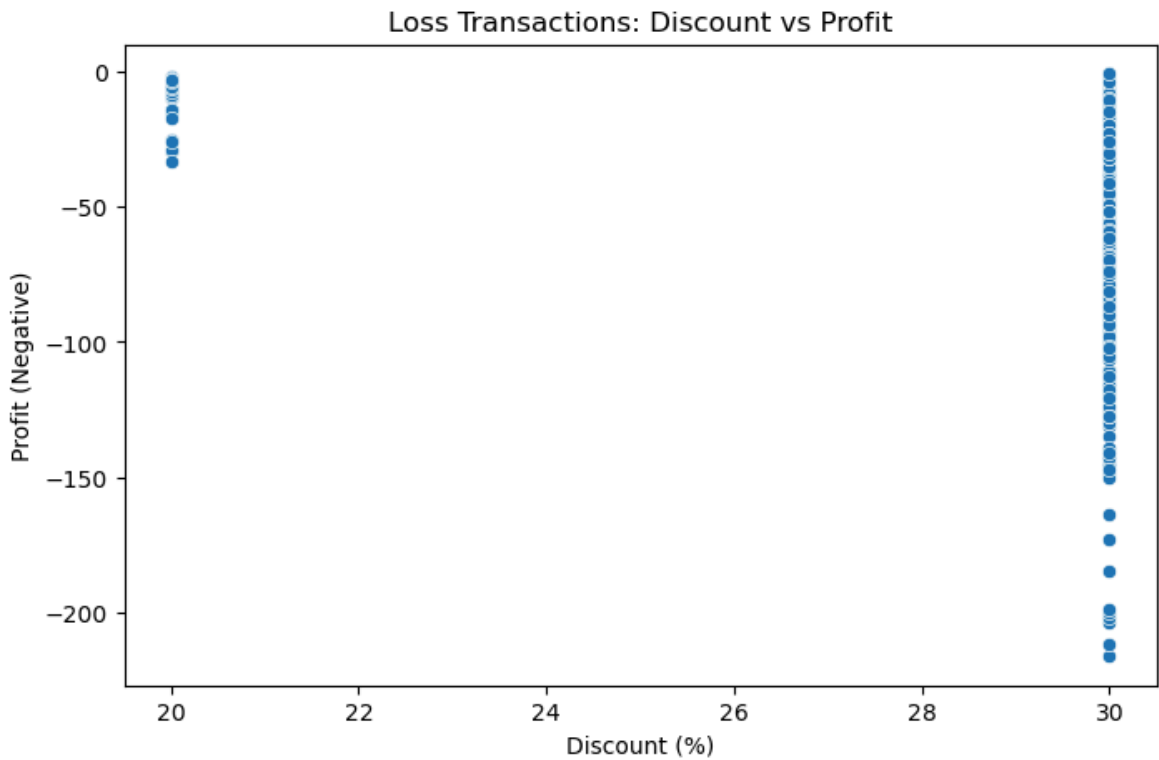
plt.title("Discount vs Profit")
plt.xlabel("Discount (%)")
plt.ylabel("Profit")

plt.axhline(0) # Profit = 0 line
plt.show()
```



```
In [35]: plt.figure(figsize=(8,5))
sns.scatterplot(x=loss['Discount_%'], y=loss['Profit_Check'])

plt.title("Loss Transactions: Discount vs Profit")
plt.xlabel("Discount (%)")
plt.ylabel("Profit (Negative)")
plt.show()
```



3. Critical Business Insight

1. Identify Loss Transactions

```
In [52]: loss = df[df['Profit_Check'] < 0]

print("Total Loss Transactions:", len(loss))
loss.head(10)
```

Total Loss Transactions: 298

Out[52]:

	Order_ID	Customer_ID	Product_ID	Category	Original_Price	Discount_%	Quantity
74	75	1075	223	Electronics	1152.10	30.0	
132	133	1007	207	Home	338.16	30.0	
145	146	1082	214	Home	1288.15	30.0	
227	228	1075	223	Electronics	1170.78	30.0	
398	399	1087	214	Home	1896.54	30.0	
463	464	1093	249	Electronics	1796.05	30.0	
541	542	1093	249	Electronics	1823.65	30.0	
676	677	1015	217	Beauty	1167.79	30.0	
698	699	1082	214	Home	1258.44	30.0	
868	869	1032	223	Electronics	1887.32	30.0	

10 rows × 21 columns



2. Check Discount Impact on Loss

```
In [53]: loss.groupby('Discount_%')['Profit_Check'].count().sort_values(ascending=False)
```

```
Out[53]: Discount_%
30.0      269
20.0       29
Name: Profit_Check, dtype: int64
```

3. Category-wise Loss Analysis

```
In [54]: loss.groupby('Category')['Profit_Check'].count().sort_values(ascending=False)
```

```
Out[54]: Category
Home          102
Electronics    90
Clothing       73
Beauty         33
Name: Profit_Check, dtype: int64
```

4. Profit Margin Calculation

```
In [64]: df['Profit_Margin'] = (df['Profit_Check'] / df['Revenue_Check']) * 100

df[['Revenue_Check', 'Profit_Check', 'Profit_Margin']].head()
```

```
Out[64]:
```

	Revenue_Check	Profit_Check	Profit_Margin
0	3869.22	2955.18	76.376634
1	1404.84	922.08	65.635944
2	3343.38	2092.42	62.583972
3	2915.04	2351.74	80.676080
4	1823.80	1415.06	77.588551

5. Set Safe Discount Rule

```
In [70]: def adjust_discount(row):
        if row['Profit_Check'] < 0:
            return max(row['Discount_%'] - 10, 0) # reduce 10%
        return row['Discount_%']

df['Adjusted_Discount'] = df.apply(adjust_discount, axis=1)
```

6. Recalculate Profit

```
In [72]: df['Original_Price'] = df['Revenue_Check'] / (1 - df['Discount_%'] / 100)
```

```
In [73]: df['Adjusted_Revenue'] = df['Original_Price'] * (1 - df['Adjusted_Discount'] / 100)
```

```
In [74]: # Cost = Revenue - Profit
df['Cost'] = df['Revenue_Check'] - df['Profit_Check']

# New Profit
df['Adjusted_Profit'] = df['Adjusted_Revenue'] - df['Cost']
```

```
In [76]: old_loss = len(df[df['Profit_Check'] < 0])
new_loss = len(df[df['Adjusted_Profit'] < 0])

print("Old Loss Count:", old_loss)
print("New Loss Count:", new_loss)
```

```
Old Loss Count: 298
New Loss Count: 15
```

```
In [78]: import matplotlib.pyplot as plt

labels = ['Before', 'After']
values = [
```

```

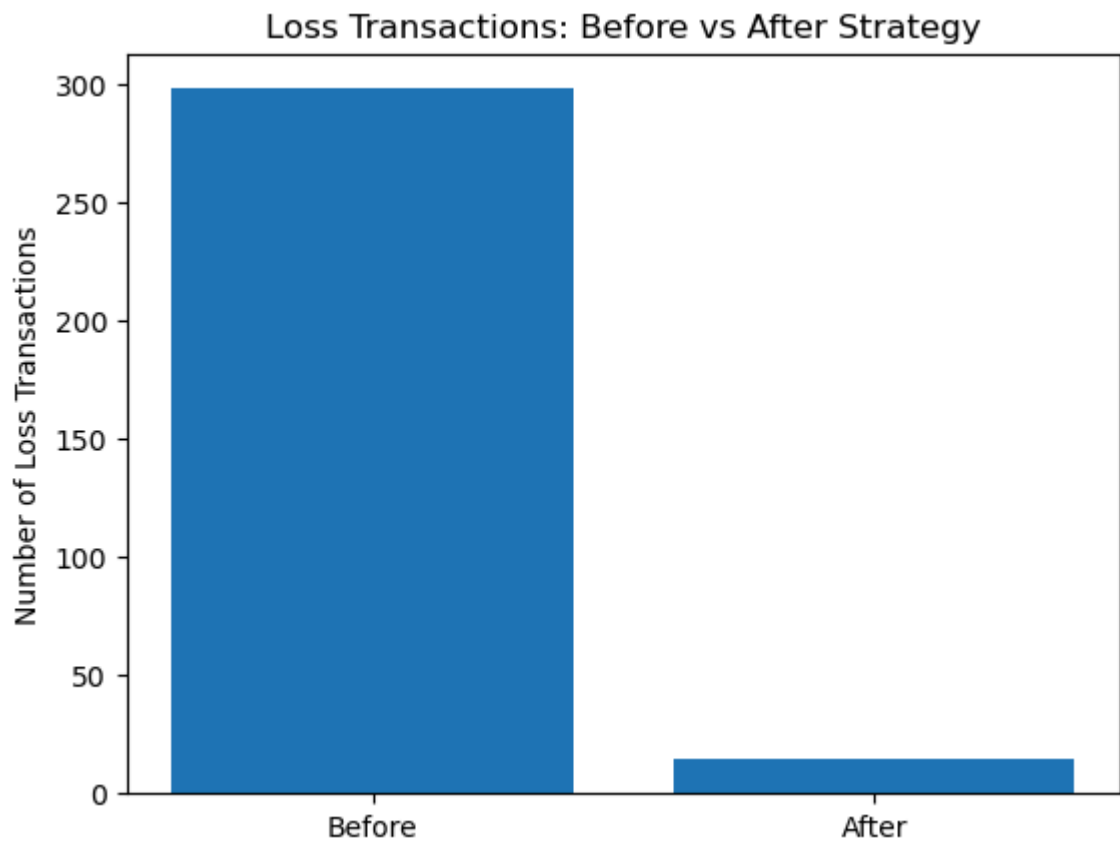
len(df[df['Profit_Check'] < 0]),
len(df[df['Adjusted_Profit'] < 0])
]

plt.bar(labels, values)

plt.title("Loss Transactions: Before vs After Strategy")
plt.ylabel("Number of Loss Transactions")

plt.show()

```



Customer Behavior Analysis

```

In [83]: customer_stats = df.groupby('Customer_ID')['Revenue_Check'].agg(['sum', 'mean', 'count'])
print(customer_stats.head(10))

```

Customer_ID	sum	mean	count
1000	928004.47	2643.887379	351
1001	639634.67	2117.995596	302
1002	709565.31	2750.253140	258
1003	448054.53	1882.582059	238
1004	578967.68	2353.527154	246
1005	307882.70	1888.850920	163
1006	403516.48	1978.021961	204
1007	587563.16	1883.215256	312
1008	380119.87	1674.536872	227
1009	453119.53	3213.613688	141

```

In [84]: import matplotlib.pyplot as plt

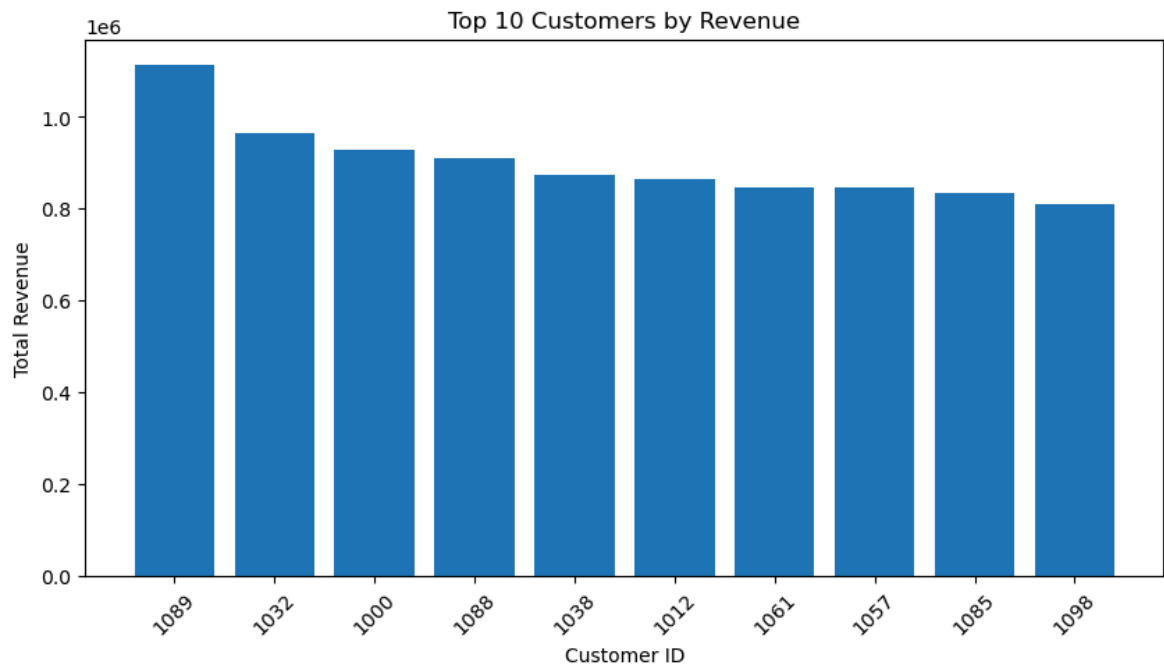
top_customers = customer_stats.sort_values(by='sum', ascending=False).head(10)

```

```
plt.figure(figsize=(10,5))
plt.bar(top_customers.index.astype(str), top_customers['sum'])

plt.title("Top 10 Customers by Revenue")
plt.xlabel("Customer ID")
plt.ylabel("Total Revenue")

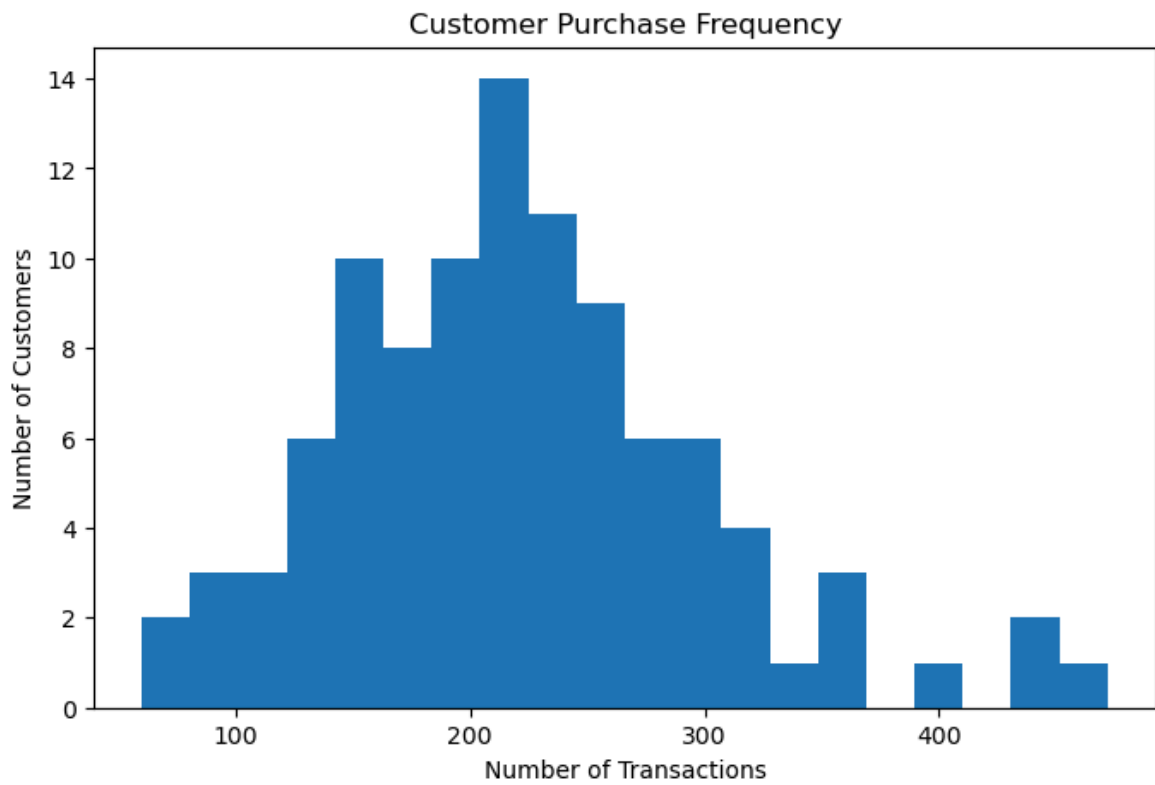
plt.xticks(rotation=45)
plt.show()
```



```
In [85]: plt.figure(figsize=(8,5))
plt.hist(customer_stats['count'], bins=20)

plt.title("Customer Purchase Frequency")
plt.xlabel("Number of Transactions")
plt.ylabel("Number of Customers")

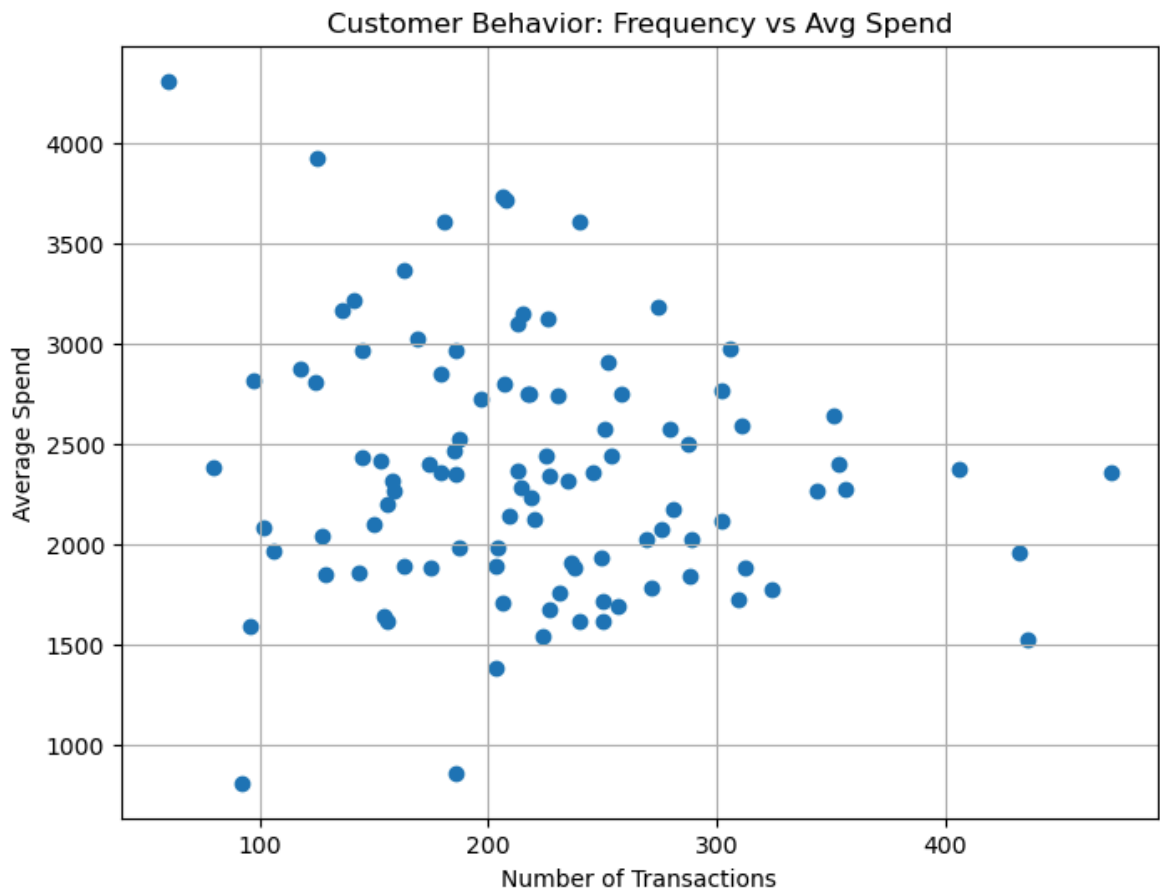
plt.show()
```



```
In [86]: plt.figure(figsize=(8,6))
plt.scatter(customer_stats['count'], customer_stats['mean'])

plt.title("Customer Behavior: Frequency vs Avg Spend")
plt.xlabel("Number of Transactions")
plt.ylabel("Average Spend")

plt.grid()
plt.show()
```



```
In [89]: # Unique customers
print("Total Customers:", df['Customer_ID'].nunique())

# Transactions per customer
customer_counts = df['Customer_ID'].value_counts()

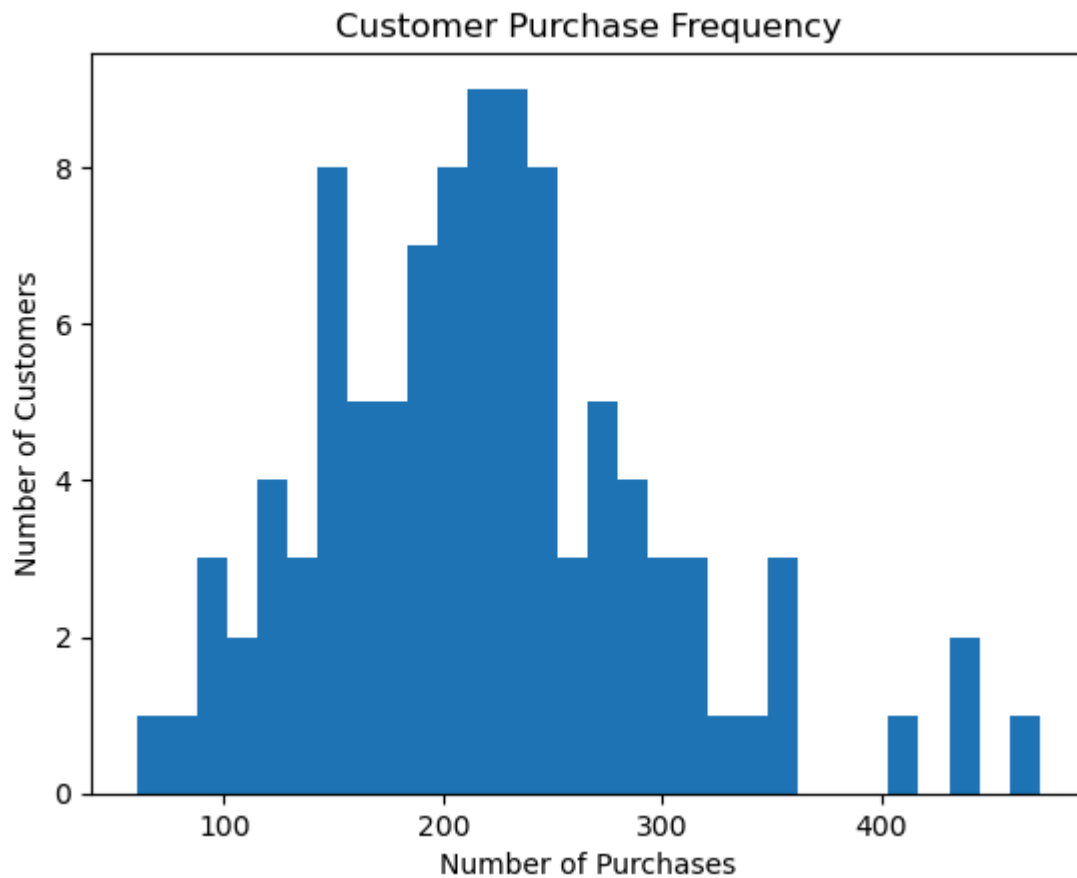
print(customer_counts.head(10))
```

```
Total Customers: 100
Customer_ID
1089    472
1091    436
1061    432
1032    406
1098    356
1057    353
1000    351
1023    344
1016    324
1007    312
Name: count, dtype: int64
```

```
In [90]: import matplotlib.pyplot as plt

plt.hist(customer_counts, bins=30)
plt.title("Customer Purchase Frequency")
plt.xlabel("Number of Purchases")
plt.ylabel("Number of Customers")

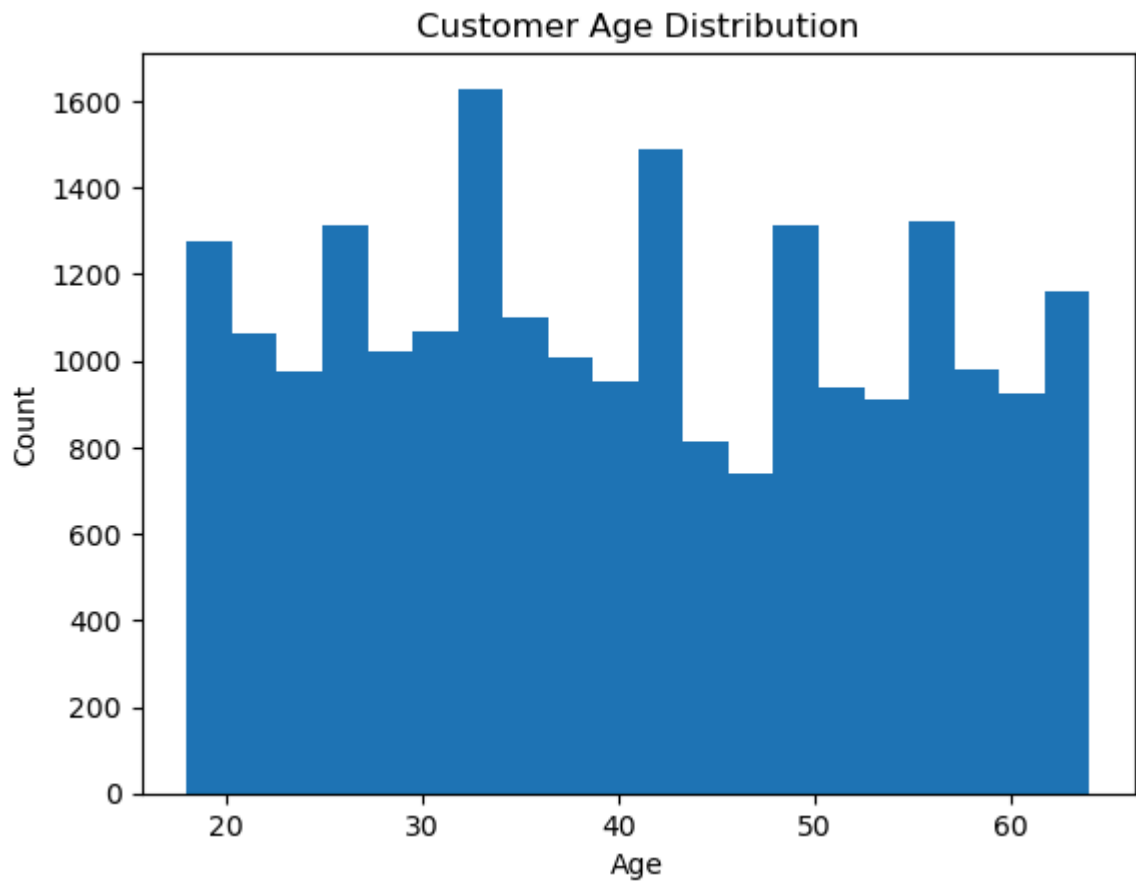
plt.show()
```



Age Analysis

```
In [93]: # Age distribution
plt.hist(df['Customer_Age'], bins=20)
plt.title("Customer Age Distribution")
plt.xlabel("Age")
plt.ylabel("Count")

plt.show()
```



Age Group	Strategy
18–25	Trendy, discounts, social media ads
26–40	Value + quality
40+	Premium, reliability

Gender-Based Spending Analysis

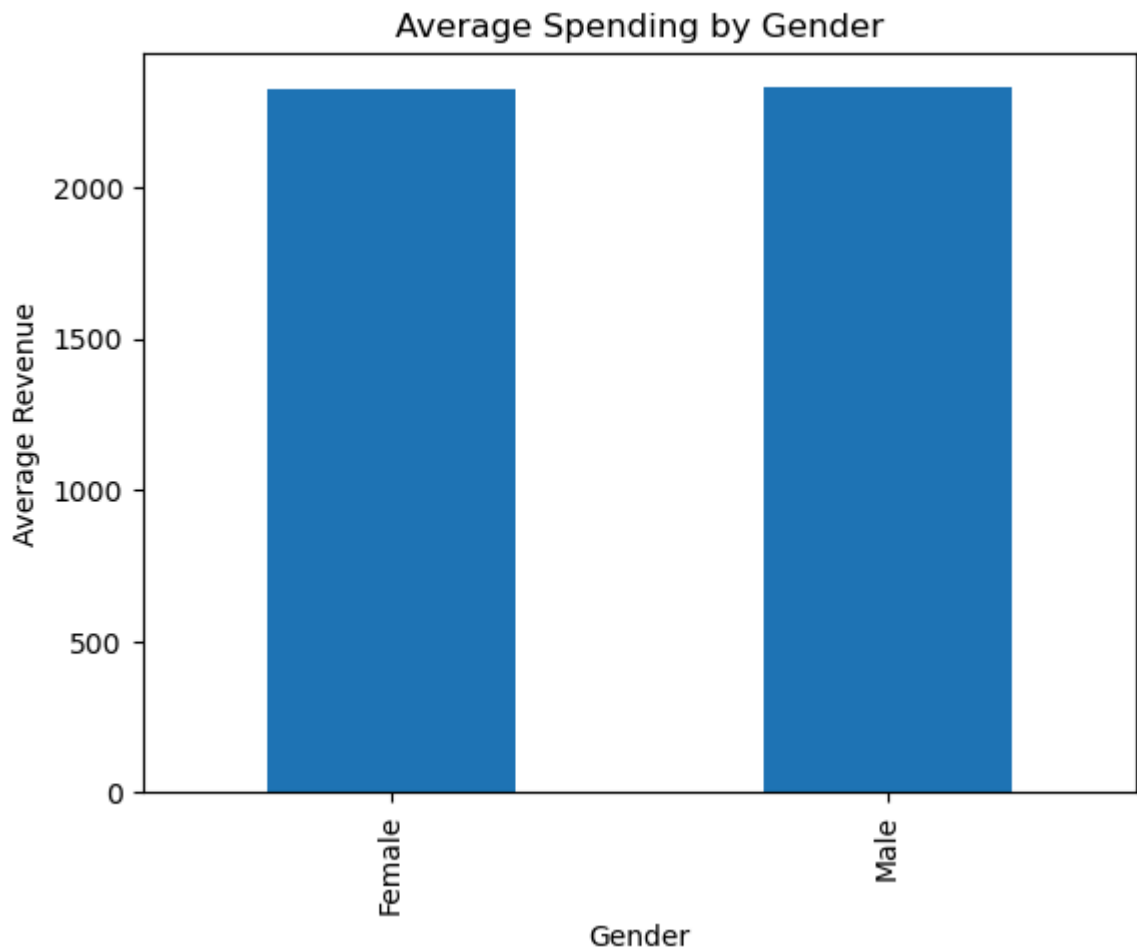
```
In [96]: gender_spend = df.groupby('Customer_Gender')['Revenue_Check'].mean()
print(gender_spend)
```

```
Customer_Gender
Female    2321.716256
Male      2326.505450
Name: Revenue_Check, dtype: float64
```

```
In [97]: gender_spend.plot(kind='bar')

plt.title("Average Spending by Gender")
plt.xlabel("Gender")
plt.ylabel("Average Revenue")

plt.show()
```



```
In [98]: # Customer total spend
customer_value = df.groupby('Customer_ID')['Revenue_Check'].sum()

# Categorize
def segment(x):
    if x > 500000:
        return "High Value"
    elif x > 200000:
        return "Medium Value"
    else:
        return "Low Value"

customer_segment = customer_value.apply(segment)

print(customer_segment.value_counts())
```

```
Revenue_Check
High Value      48
Medium Value    48
Low Value       4
Name: count, dtype: int64
```

Segment	Action
High Value	VIP treatment
Medium	Upsell
Low	Discounts

Regional Analysis

```
In [110... print(df['Region'].value_counts())
```

```
Region
West      5930
East      5651
South     5526
North     4893
Name: count, dtype: int64
```

```
In [111... # Revenue by Region
```

```
In [112... region_revenue = df.groupby('Region')['Revenue_Check'].sum()
print(region_revenue)
```

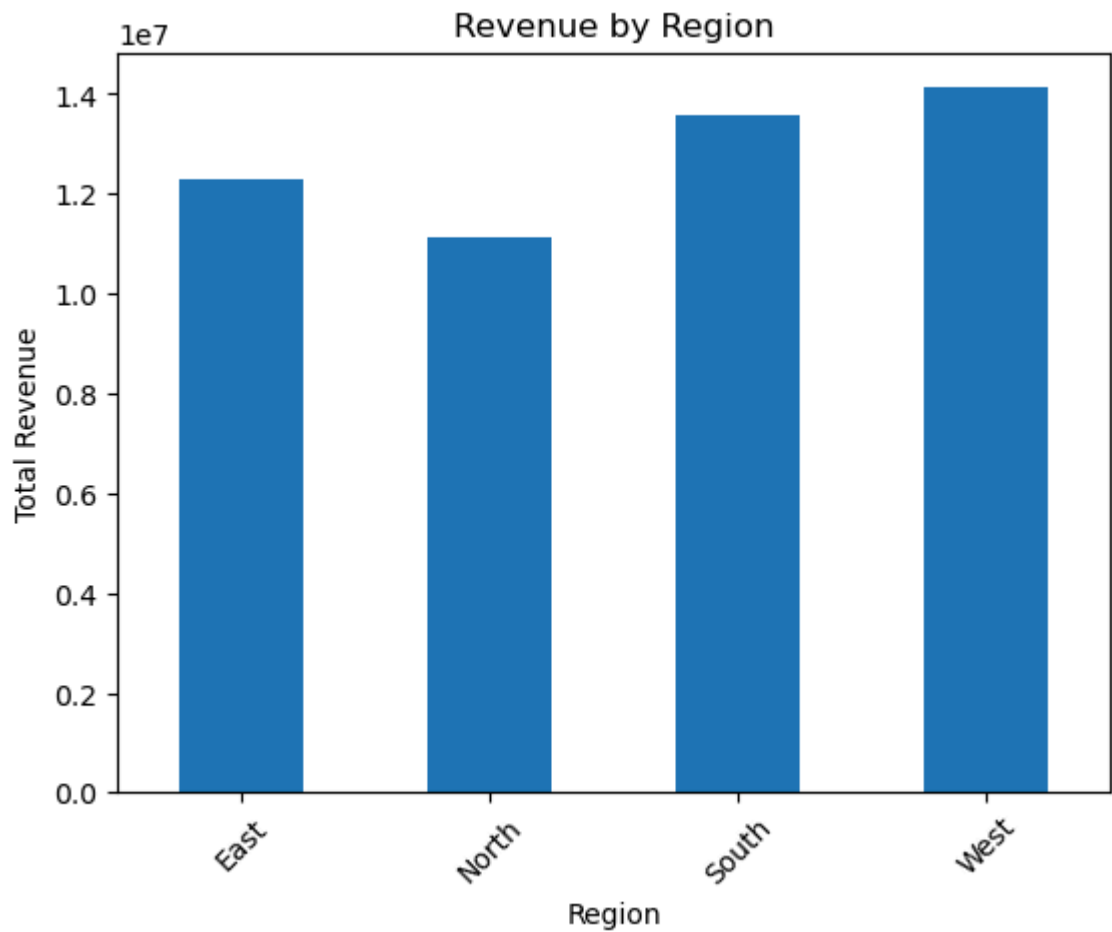
```
Region
East      12282885.98
North     11141599.52
South     13583417.30
West      14123402.81
Name: Revenue_Check, dtype: float64
```

```
In [113... import matplotlib.pyplot as plt
```

```
region_revenue.plot(kind='bar')
```

```
plt.title("Revenue by Region")
plt.xlabel("Region")
plt.ylabel("Total Revenue")
```

```
plt.xticks(rotation=45)
plt.show()
```



```
In [114... # Profit by Region
```

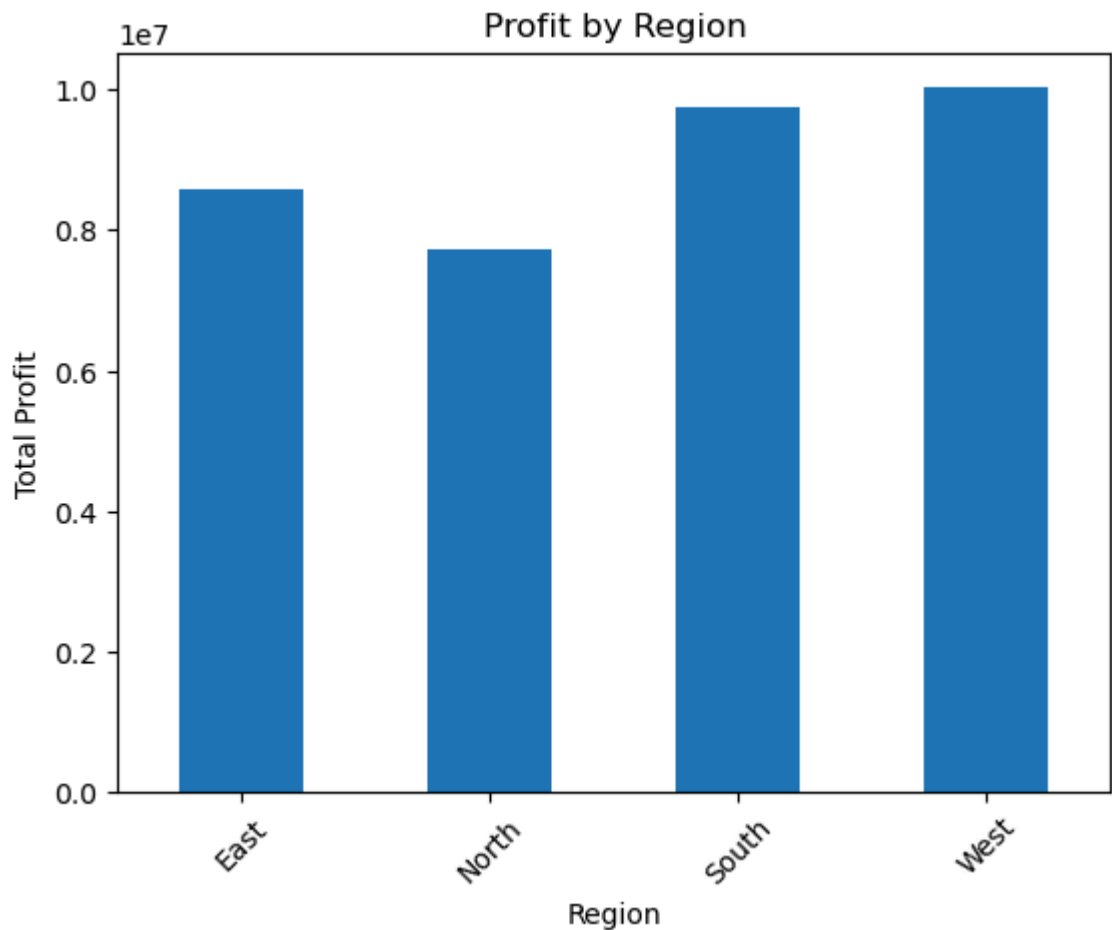
```
In [115... region_profit = df.groupby('Region')['Profit_Check'].sum()
print(region_profit)
```

```
Region
East      8.591750e+06
North     7.730052e+06
South     9.750726e+06
West     1.001971e+07
Name: Profit_Check, dtype: float64
```

```
In [116... region_profit.plot(kind='bar')

plt.title("Profit by Region")
plt.xlabel("Region")
plt.ylabel("Total Profit")

plt.xticks(rotation=45)
plt.show()
```



```
In [117...] # Loss Analysis by Region
```

```
In [118...] loss_region = df[df['Profit_Check'] < 0].groupby('Region').size()
print(loss_region)
```

```
Region
East      55
North    102
South     68
West      73
dtype: int64
```

```
In [119...] # Average Order Value
```

```
In [120...] region_avg = df.groupby('Region')['Revenue_Check'].mean()
print(region_avg)
```

```
Region
East      2173.577416
North     2277.048747
South     2458.092164
West      2381.686815
Name: Revenue_Check, dtype: float64
```

```
In [123...] # Combine Region + Category
```

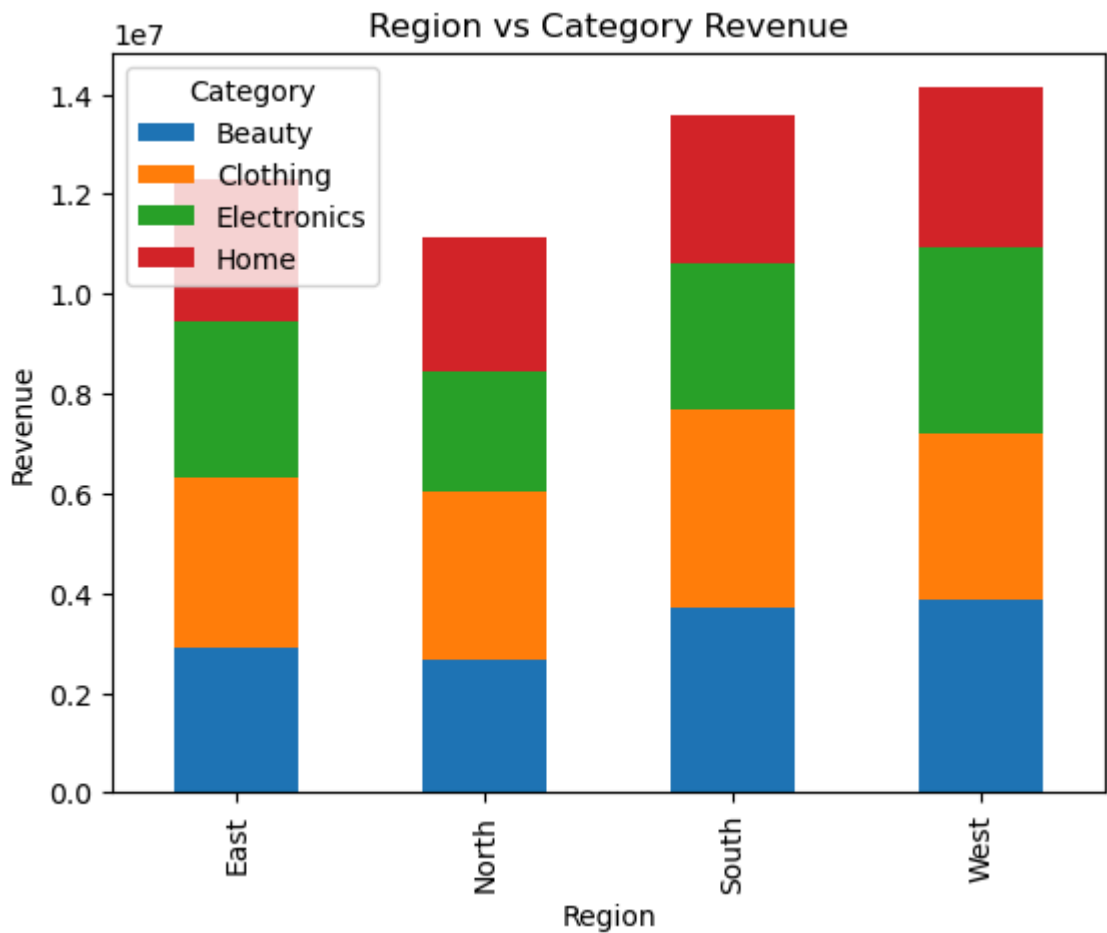
```
In [121...] region_category = df.groupby(['Region', 'Category'])['Revenue_Check'].sum().unsta
print(region_category)
```

Category	Beauty	Clothing	Electronics	Home
Region				
East	2894032.85	3433792.70	3134821.73	2820238.70
North	2658563.77	3387473.10	2413655.11	2681907.54
South	3715269.05	3987246.68	2921021.18	2959880.39
West	3867735.77	3337997.86	3728820.67	3188848.51

```
In [122... region_category.plot(kind='bar', stacked=True)

plt.title("Region vs Category Revenue")
plt.xlabel("Region")
plt.ylabel("Revenue")

plt.show()
```



```
In [125... region_avg_profit = df.groupby('Region')['Profit_Check'].mean()
print(region_avg_profit)
```

```
Region
East    1520.394575
North   1579.818429
South   1764.517901
West    1689.664268
Name: Profit_Check, dtype: float64
```

Payment Method Insights

```
In [126... df['Payment_Method'].value_counts()
```

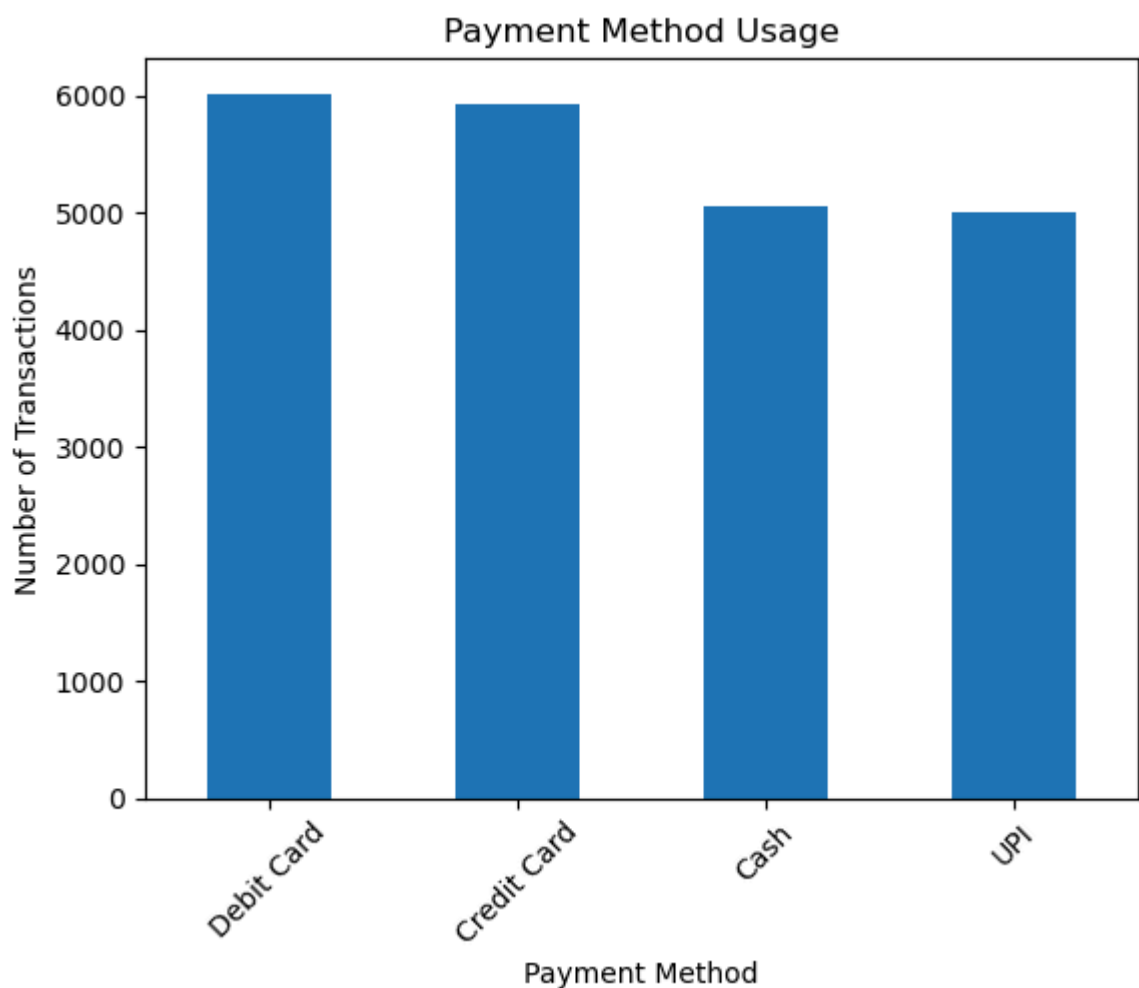
```
Out[126...] Payment_Method
Debit Card    6012
Credit Card   5925
Cash          5055
UPI           5008
Name: count, dtype: int64
```

```
In [127...] import matplotlib.pyplot as plt

df['Payment_Method'].value_counts().plot(kind='bar')

plt.title("Payment Method Usage")
plt.xlabel("Payment Method")
plt.ylabel("Number of Transactions")

plt.xticks(rotation=45)
plt.show()
```



```
In [128...] payment_revenue = df.groupby('Payment_Method')['Revenue_Check'].sum()
print(payment_revenue)
```

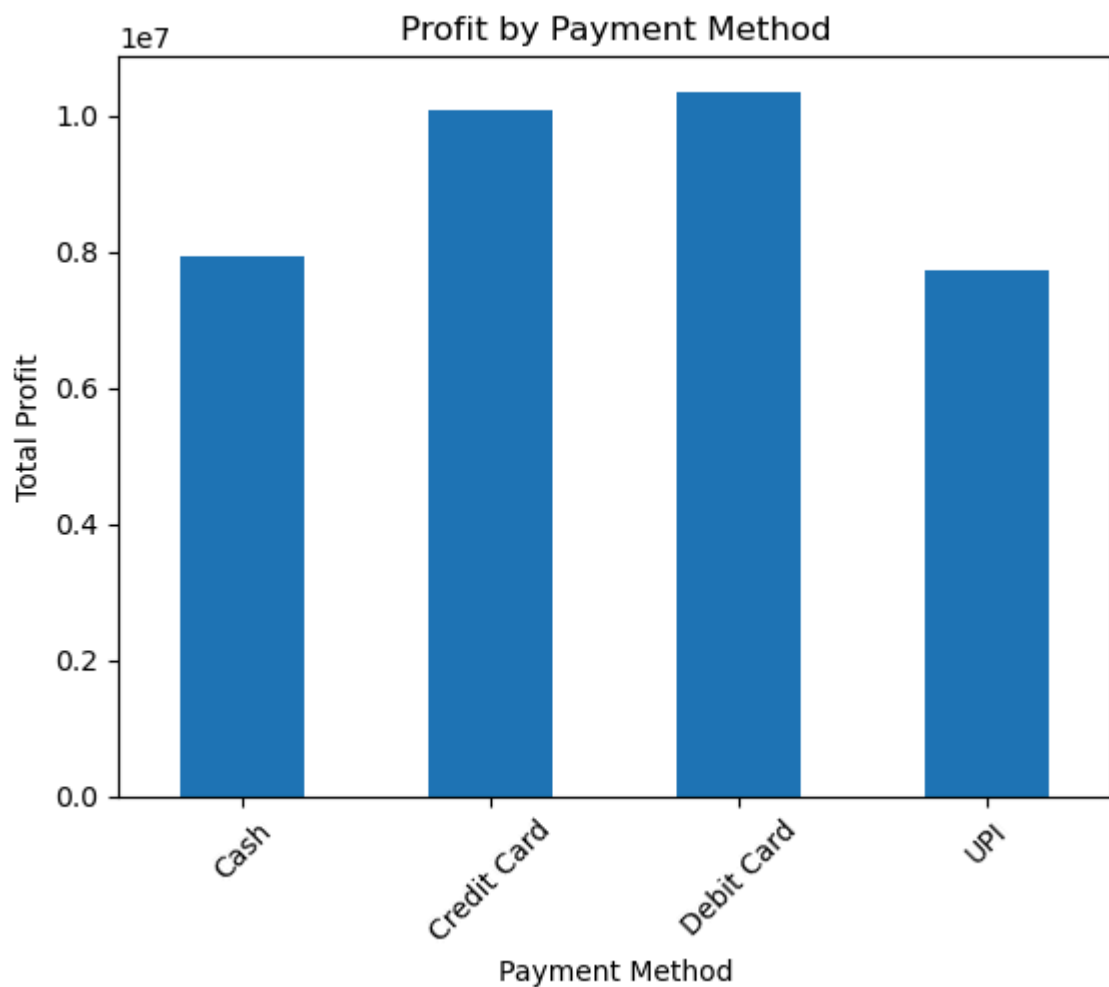
```
Payment_Method
Cash          11154349.07
Credit Card   14334119.18
Debit Card     14523389.11
UPI            11119448.25
Name: Revenue_Check, dtype: float64
```

```
In [129...] payment_profit = df.groupby('Payment_Method')['Profit_Check'].sum()
```

```
print(payment_profit)
```

```
Payment_Method  
Cash          7.923709e+06  
Credit Card  1.009325e+07  
Debit Card    1.035087e+07  
UPI           7.724409e+06  
Name: Profit_Check, dtype: float64
```

```
In [130... payment_profit.plot(kind='bar')  
  
plt.title("Profit by Payment Method")  
plt.xlabel("Payment Method")  
plt.ylabel("Total Profit")  
  
plt.xticks(rotation=45)  
plt.show()
```



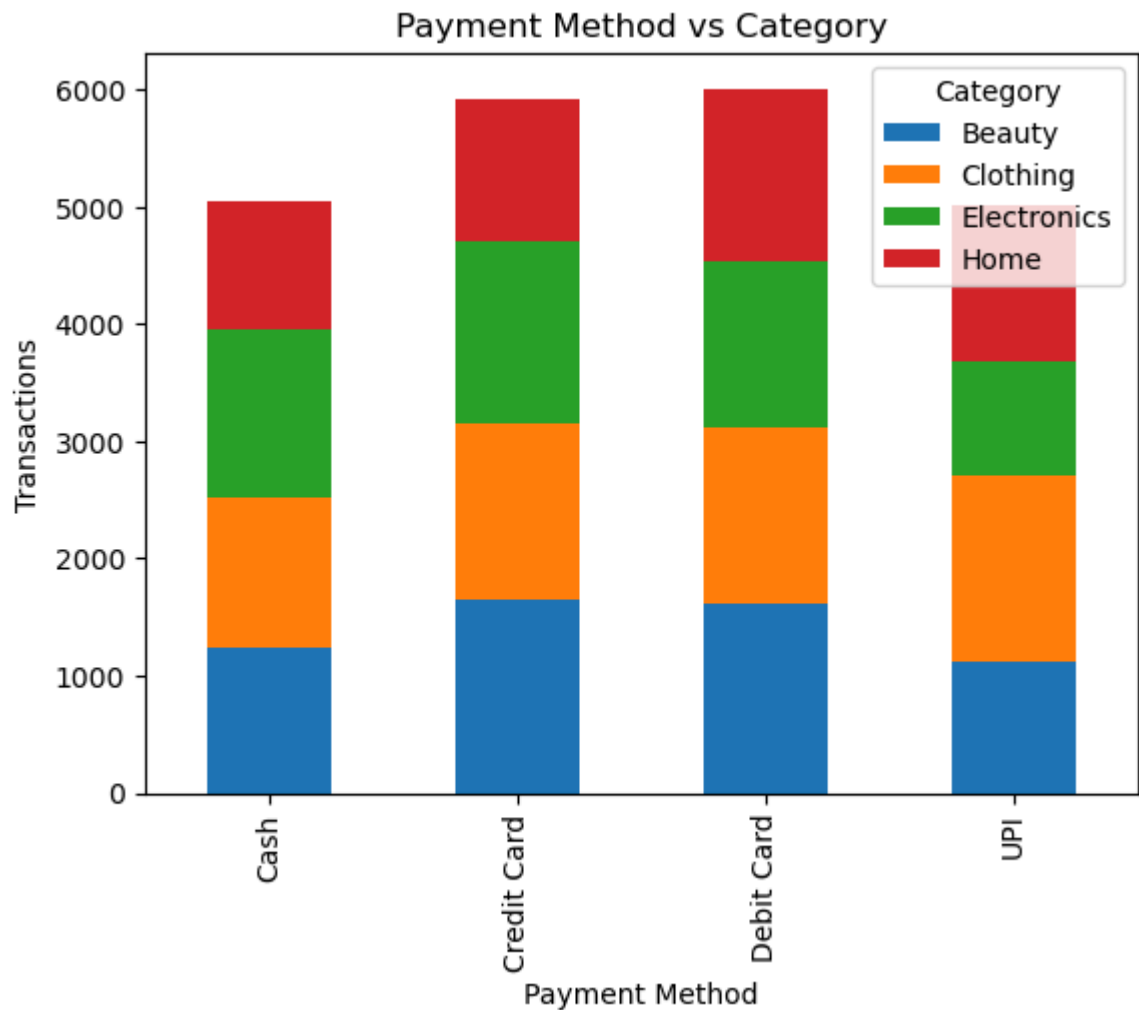
```
In [132... payment_category = df.groupby(['Payment_Method', 'Category']).size().unstack()  
print(payment_category)
```

```
Category      Beauty  Clothing  Electronics  Home  
Payment_Method  
Cash          1242      1271          1436  1106  
Credit Card   1657      1489          1562  1217  
Debit Card     1620      1503          1419  1470  
UPI            1125      1592           973  1318
```

```
In [133... payment_category.plot(kind='bar', stacked=True)
```

```
plt.title("Payment Method vs Category")
plt.xlabel("Payment Method")
plt.ylabel("Transactions")

plt.show()
```



```
In [134... category_revenue = df.groupby('Category')['Revenue_Check'].sum()
print(category_revenue)
```

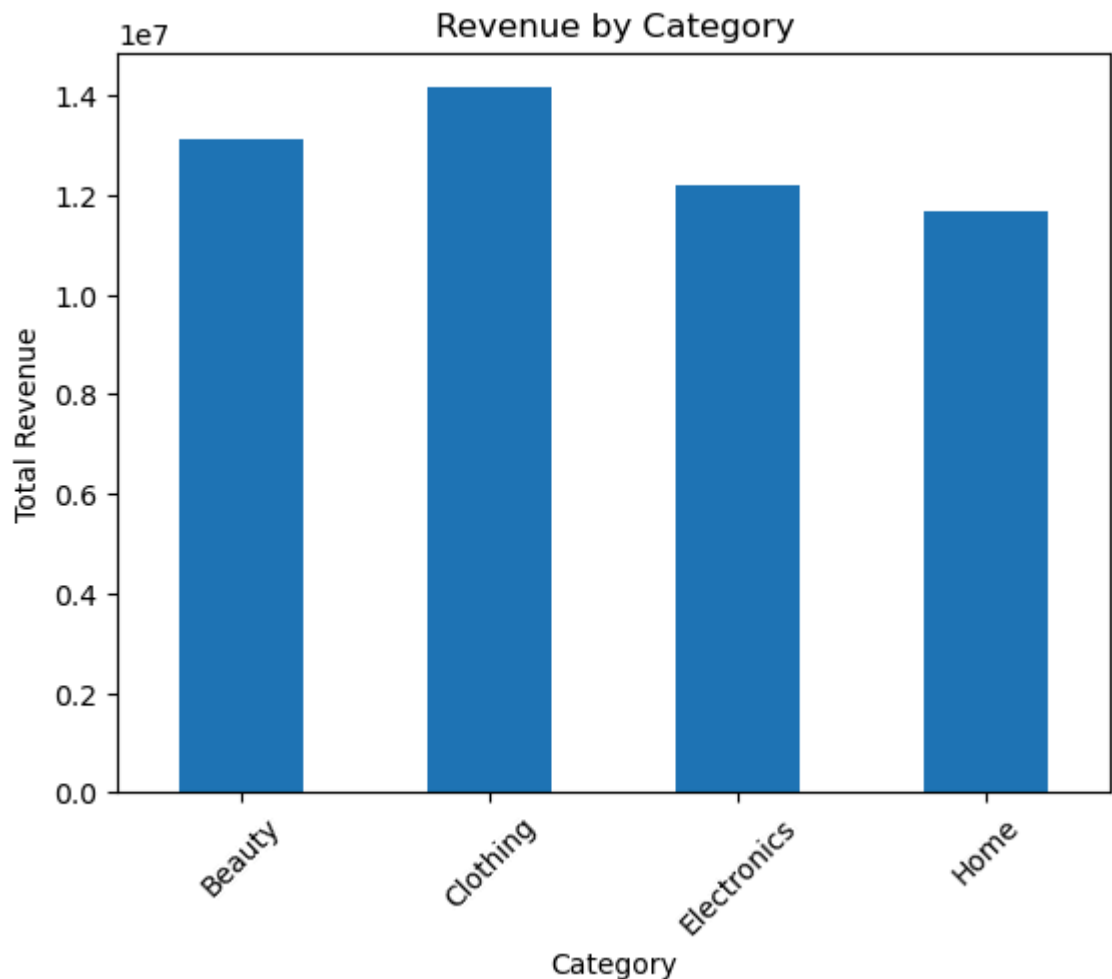
```
Category
Beauty      13135601.44
Clothing     14146510.34
Electronics  12198318.69
Home         11650875.14
Name: Revenue_Check, dtype: float64
```

```
In [135... import matplotlib.pyplot as plt

category_revenue.plot(kind='bar')

plt.title("Revenue by Category")
plt.xlabel("Category")
plt.ylabel("Total Revenue")

plt.xticks(rotation=45)
plt.show()
```



```
In [136... category_profit = df.groupby('Category')['Profit_Check'].sum()
print(category_profit)
```

```
Category
Beauty      9.375043e+06
Clothing    9.972617e+06
Electronics 8.473511e+06
Home        8.271065e+06
Name: Profit_Check, dtype: float64
```

```
In [137... profit_margin = (category_profit / category_revenue) * 100
print(profit_margin)
```

```
Category
Beauty      71.371249
Clothing    70.495247
Electronics 69.464582
Home        70.990934
dtype: float64
```

```
In [139... category_region = df.groupby(['Category', 'Region'])['Revenue_Check'].sum().unsta
print(category_region)
```

```
Region      East      North      South      West
Category
Beauty      2894032.85  2658563.77  3715269.05  3867735.77
Clothing    3433792.70  3387473.10  3987246.68  3337997.86
Electronics 3134821.73  2413655.11  2921021.18  3728820.67
Home        2820238.70  2681907.54  2959880.39  3188848.51
```

